

Multi-Agent Extraction System — Technical Deep Dive

LangGraph Orchestration, Specialist Agent Pipelines, Reconciliation & Validation Logic

Architecture: LangGraph Parallel Multi-Agent Workflow	Output: 20 Canonical Fields with Provenance Citations
Agent Framework: StateGraph with Custom Reducers & Feedback Loops	LLM / Extraction Engine: Groq Qwen 27B + Evidence Grounding

1. Multi-Agent System Architecture Overview

Single-prompt Large Language Model extraction fails on complex RFP procurement packages because a single context window cannot reliably process 100+ pages, preserve tabular specifications, resolve legal amendments across addenda, and verify factual citations. Our platform solves this using a **LangGraph Stateful Multi-Agent System**. Instead of dumping raw PDFs into an LLM, the system breaks extraction into specialized domain sub-tasks, coordinates them via a centralized state board (RFPState), executes parallel retrieval over our Hybrid Search Engine, resolves contractual overrides via an Addendum Reconciler, and enforces strict evidence grounding with an automated Critic/Validator.

Workflow Stage	Node Name	Agent Responsibility & Code Logic	State Transition
1. Orchestration	Orchestrator	Initializes RFPState, binds bid identifier, and fans out execution to 4 specialist agents in parallel.	START → Specialists
2. Parallel Specialists	entity_specialist logistics_specialist product_specialist legal_specialist	Specialist agents query the search tool for domain queries, collect candidate chunks, and prompt Groq LLM for field extraction.	Specialists → addendum_reconciler (Fan-In)
3. Addendum Reconciler	addendum_reconciler	Fetches all addenda chunks, detects timeline and requirement modifications, overrides outdated fields, and logs audit diffs.	addendum_reconciler → validator
4. Validator & Critic	validator	Audits all 20 fields against authentic chunk IDs in EvidenceStore. Rejects ungrounded claims or invalid citations.	validator → should_retry?
5. Feedback / Retry	retry_agent	Executes targeted re-search with expanded queries for rejected fields (max 2 retries). Feeds back into validator.	retry_agent → validator (Loop)
6. Final Serializer	serializer	Compiles canonical BidOutput model, aliases field names, and writes final validated JSON outputs.	serializer → END

2. Shared State Management (RFPState)

Code Module: app/graph/state.py

LangGraph coordinates agents using a strongly-typed shared memory object. Because specialist agents run concurrently, we define custom **reducer functions** that merge results from parallel branches without race conditions:

```
class RFPState(TypedDict):
    bid_id: str
    extracted_fields: Annotated[Dict[str, FieldResult], merge_field_results] # Merges parallel agent outputs
    addendum_changes: Annotated[List[AddendumChange], append_changes] # Accumulates addendum diffs
    validation_result: Optional[ValidationResult] # Critic audit report
    retry_count: int # Loop counter (max 2)
    final_output: Optional[BidOutput] # Serialized 20-field model
    errors: Annotated[List[str], append_strings] # Trace and error logs
```

3. Specialist Agent Extraction Pipelines & Logic

The 20 required procurement fields are partitioned into 4 specialist agent domains. Each specialist agent executes a standardized 4-step pipeline: **(1) Targeted Query Formulation** → **(2) Tool-based Evidence Search** → **(3) Structured LLM Grounding** → **(4) FieldResult Construction**.

Agent 1: Entity & Contact Specialist (app/agents/entity_agent.py)

- **Assigned Fields:** bid_number, title, company_name, contact_info, bid_summary.
- **Targeted Queries:** 'official bid solicitation RFP number', 'issuing organization agency school district', 'procurement contact email phone'.
- **Real Extraction Logic (Bid 1):** Retrieves the Dallas ISD Bonfire portal HTML and RFP cover page. Extracts bid_number: 'JA-207652' and company_name: 'Dallas Independent School District' with chunk citations.
- **Bid Summary Generation:** Synthesizes a concise 3-5 sentence procurement overview grounded strictly in the retrieved scope passages.

Agent 2: Dates & Logistics Specialist (app/agents/logistics_agent.py)

- **Assigned Fields:** due_date, bid_submission_type, term_of_bid, pre_bid_meeting, delivery_date.
- **Targeted Queries:** 'due date closing deadline schedule', 'bid submission portal electronic sealed', 'delivery window calendar days after award', 'pre-bid conference mandatory date time'.
- **Real Extraction Logic (Bid 2):** Searches PORFP_-_Dell_Laptop_Final.pdf. Extracts due_date: 'July 2, 2024 at 2:00 PM EST', bid_submission_type: 'Electronic via eMMA portal', and delivery_date: 'Within 45 calendar days after contract award'.

Agent 3: Product & Technical Specs Specialist (app/agents/product_agent.py)

- **Assigned Fields:** product, model_no, part_no, product_specification, mfg_for_registration, installation.
- **Targeted Queries:** 'product name quantity required devices', 'Dell model number SKU part number', 'technical specifications CPU processor RAM storage display', 'manufacturer authorization partner registration'.
- **Real Extraction Logic (Bid 2):** Searches Dell_Laptop_Specs.pdf and extracts Markdown tables. Populates model_no: 'Dell Latitude 5550', part_no: '210-BLMX', and product_specification: 'Intel Core Ultra 5 125U (12 cores), 16GB DDR5 RAM, 256GB SSD, 15.6 FHD Display, 3-Year Extended Warranty'.

Agent 4: Legal & Commercial Specialist (app/agents/legal_agent.py)

- **Assigned Fields:** bid_bond_requirement, payment_terms, any_additional_documentation_required, contract_or_cooperative_to_use.
- **Targeted Queries:** 'bid bond surety proposal security deposit percentage', 'payment terms net invoicing', 'required affidavits forms certificates compliance', 'state master contract cooperative vehicle'.
- **Real Extraction Logic (Bid 2):** Reads Contract_Affidavit.pdf and Mercury_Affidavit.pdf. Extracts any_additional_documentation_required: 'Contract Affidavit and Mercury Affidavit' and accurately records bid_bond_requirement: 'Not required' (zero hallucination).

4. Addendum Reconciliation Agent & Override Logic

Code Module: app/reconciliation/reconciler.py

The Procurement Problem: In government and school RFPs, addenda legally supersede original contract terms. If an AI only reads the base RFP, it reports outdated deadlines and incorrect specifications.

How the Addendum Reconciler Operates:

1. **Addenda Harvesting:** Queries Qdrant for all chunks where document_type = 'addendum' ordered sequentially by addendum_number.
2. **Modification Analysis:** Passes the current draft fields and addenda text into a specialized reconciliation prompt.
3. **Override Execution:** When an amendment alters a field, the Reconciler overwrites the value, attaches the addendum chunk citation, and creates an AddendumChange record.

• **Concrete Bid 1 Walkthrough:**

- *Base RFP Draft:* Due Date = June 25, 2024 at 2:00 PM CST (from JA-207652 FINAL.pdf).
- *Addendum 2 Clause:* 'The proposal due date has been extended to July 9, 2024 at 2:00 PM CST.'
- *Reconciled State:* Due Date: 'July 9, 2024 at 2:00 PM CST', source citation updated to Bid_1_Addendum_2.pdf (Page 1), audit note: 'Extended by Addendum 2 (original date: June 25, 2024)'.

5. Validator / Critic Agent & Targeted Retry Loop

Code Modules: app/validation/validator.py, app/agents/retry_agent.py, app/graph/nodes/retry.py

To eliminate hallucinations and enforce zero-defect extraction, the **Validator Critic** audits every field before allowing the pipeline to finish.

Deterministic Audit Rule	Validation Logic & Criteria	Action if Violated (Critic Action)
Rule 1: No Evidence → No Value	If value != None, len(sources) must be ≥ 1\$. A claim without source evidence is rejected.	Flags MISSING_CITATION → routes field to Retry Agent.
Rule 2: Chunk Registry Provenance	Every cited chunk_id must exist in the local EvidenceStore database.	Flags UNKNOWN_CHUNK → forces re-extraction from verified chunks.
Rule 3: Confidence Calibration	Confidence must be in [0.0, 1.0]\$. If value is None, confidence must be ≥ 0.20\$.	Flags INVALID_CONFIDENCE → resets confidence score.
Rule 4: Citation Validity	Citations must have valid non-empty file_name and page_number ≥ 1.	Flags INVALID_PAGE → triggers metadata repair.

The Retry Feedback Loop in Action:

When the Validator rejects a field, the conditional edge should_retry evaluates the state:

- If not validation_result.is_valid and retry_count < 2 → routes to retry_agent.
- The RetryAgent expands the search query with domain synonyms (e.g. 'all details specifications for bid bond surety') and queries the search tool with increased depth (top_k=6).
- Re-extracted fields are merged back into RFPState and routed **back to the Validator** for re-auditing.
- If still unverified after 2 retries, the field is safely resolved to value: null with 'Not found in documents.'

6. Serialized Canonical Output Schema

Code Module: app/graph/nodes/serializer.py, app/extraction/schemas.py

The Serializer formats the final state into the exact JSON schema required by Section 8.1 of the assignment specification:

```
{
  "bid_id": "Bid1",
  "fields": {
    "Due Date": {
      "value": "July 9, 2024 at 2:00 PM CST",
      "sources": [{"file": "Bid_1_Addendum_2.pdf", "page": 1, "chunk_id": "Bid1::Addendum_2.pdf::p1::chk_01"}],
      "confidence": 0.95,
      "notes": "Extended by Addendum 2 (original date: June 25, 2024)"
    },
    "Bid Bond Requirement": {
      "value": null,
      "sources": [],
      "confidence": 0.0,
      "notes": "Not found in documents."
    }
  },
  "addendum_changes": [{"field": "due_date", "change_type": "MODIFICATION", "new_value": "July 9, 2024 at 2:00 PM CST"}],
  "validation": {"passed": 19, "failed": 0, "not_found": 1}
}
```

Summary: The LangGraph multi-agent extraction pipeline delivers deterministic precision, automatic addendum reconciliation, and strict evidence verification for all 20 canonical procurement fields.